

Cohort 3.0

1 Which statement is correct?

- ☐ A. Function declarations and function expressions are exactly the same.
- ☒ B. Function declarations are hoisted, while function expressions are not hoisted in the same way. Correct Selection
- ☐ C. Function expressions are hoisted, but function declarations are not.
- ☐ D. Neither is hoisted.

2 Which statement is technically correct?

- ☐ A. Parameters and arguments are the same thing.
- ☐ B. Parameters are values passed during a function call.
- ☐ C. Arguments are placeholders in a function definition.
- ☒ D. Parameters are placeholders, while arguments are actual values passed to the function. Correct Selection

3 Which situation is generally NOT recommended for arrow functions?

- ☐ A. Utility/helper functions
- ☐ B. Single-expression functions
- ☒ C. Object methods that rely on this Correct Selection
- ☐ D. Callback functions

4 When is a default parameter value used?

- ☐ A. When any falsy value is passed.
- ☒ B. Only when the argument is undefined or omitted. Correct Selection
- ☐ C. When null is passed.
- ☐ D. When an empty string is passed.

5 Which statement about rest parameters is true?

- ☐ A. Rest parameters must always be the first parameter.
- ☐ B. A function can have multiple rest parameters.
- ☒ C. Rest parameters collect remaining arguments into an array. Correct Selection
- ☐ D. Rest parameters store arguments as an object.

6 Which statement about return is true?

☐ A. A function can execute multiple return statements in a single call.

☐ B. return pauses a function and resumes later.

☒ C. Once a return executes, the function immediately exits.

Correct Selection

☐ D. return can only return primitive values.

7 Which capability directly results from functions being first-class citizens?

☐ A. Functions can only be declared globally.

☒ B. Functions can be stored in variables and passed to other functions.

Correct Selection

☐ C. Functions automatically become callbacks.

☐ D. Functions cannot be returned from other functions.

8 Every callback function is:

☐ A. Recursive

☐ B. Anonymous

☒ C. Passed to another function for later execution

Correct

☒ D. A higher-order function

Your Answer

9 Which statement is true?

☐ A. Every callback is a higher-order function.

☐ B. Every higher-order function is a callback.

☒ C. A function that accepts another function or returns one is a higher-order function.

Correct Selection

☐ D. Higher-order functions must be recursive.

10 Why are anonymous functions commonly used as callbacks?

☐ A. They execute faster.

☐ B. They reduce memory usage.

☒ C. They are often needed only once and don't require a name.

Correct

☒ D. JavaScript requires callbacks to be anonymous.

Your Answer

11 Historically, what was the primary purpose of IIFEs?

☐ A. Improving recursion performance

☒ B. Creating private scope and avoiding global variable pollution

Correct Selection

☐ C. Replacing loops

☐ D. Making functions asynchronous

12 Which characteristic is NOT required for a function to be pure?

☐ A. Same input should produce the same output.

☐ B. No side effects.

☒ C. Must use only local variables.

Correct

☒ D. Predictable behavior.

Your Answer

13 A function becomes impure if it:

☐ A. Uses parameters.

☐ B. Uses loops.

☒ C. Modifies external state or depends on changing external data.

Correct Selection

☐ D. Returns an object.

14 Which condition is essential for recursion to terminate?

☐ A. Callback function

☒ B. Base case

Correct Selection

☐ C. Arrow function

☐ D. Rest parameter

15 Which description best represents recursion?

☐ A. Solving a problem by repeatedly creating larger versions of it.

☒ B. Solving a problem by reducing it into smaller versions of the same problem.

Correct Selection

☐ C. Executing multiple functions simultaneously.

☐ D. Repeating code without function calls.

Highest Scoring Student

easy 50 Points Array 3.0

Problem

You are given an array of student objects. Each student has a name and marks.

Return the name of the student who has the highest marks.

If multiple students have the same highest marks, return the first one.

Code Editor

Language javascript

Value: 279 chars

Reset Boilerplate

```
1 function highestScorer(students) {  
2     // Write your code here  
3     let highest = students[0]  
4     for (let mark of students) {  
5         if (highest.marks < mark.marks) {  
6             highest = mark;  
        }
```

Input Format

N = number of students. Each of the next N lines contains a student's name and marks. Input: 3 Aman
85 Priya 92 Raj 78

Output Format

Name of the highest scoring student Output : priya

Constraints

- $1 \leq N \leq 100000$
- $1 \leq \text{name.length} \leq 50$
- $0 \leq \text{marks} \leq 100$

Result

Wrong Answer

Score: 0

Attempts: 1

Language: javascript

Testcase

Result

Visible: 2

Hidden: 0

Submission Summary

All Tests Passed

Points: 50/50

Tests: 5/5

Accepted

Time: 0.20

Memory: 34488.00

Run

Submit

Count Word Frequency

easy

50 Points

Array 3.0

Problem

Given a string text, return an object containing the frequency of each word.

Words are case-insensitive.

Input Format

A single line containing a string text Input: JavaScript is fun

Output Format

return an object/dictionary where: key = word (in lowercase) value = frequency of that word Output:
javascript: 1, is: 1, fun: 1

Constraints

- $1 \leq \text{text.length} \leq 10^5$
- Text contains only: - English letters (a-z, A-Z)
- Spaces Words are separated by single or multiple spaces.

Result

failed

Score: 0

Code Editor

Language

javascript

Value: 322 chars

Reset Boilerplate

```
1 function wordFrequency(text) {
2     // Write your code here
3     let str = text.toLowerCase().split(" ");
4     let obj = {};
5
6     for (let word of str) {
7         if (obj[word]) {
8             obj[word]++;
9         }
10        else {
11            obj[word] = 1;
12        }
13    }
14    return obj;
15 }
16
17 module.exports = { wordFrequency };
```

Testcase

Result

Visible: 2

Hidden: 0

Submission Summary

All Tests Passed

Points: 50/50

Tests: 5/5

Accepted

Time: 0.20

Memory: 36068.00

Run

Submit

Find Missing Product

easy

50 Points

Array 3.0

Problem

You are given two arrays.

products contains all product IDs.

sold contains IDs of sold products.

Return an array of product IDs that were never sold.

Input Format

Code Editor

Language

javascript

Value: 289 chars

Reset Boilerplate

```
1 function unsoldProducts(products, sold) {
2     // Write your code here
3     let combine = products.concat(sold);
4     let unique = combine.filter((value, index) =>
5         combine.indexOf(value) === combine.lastIndexOf(value)
6     );
7     return unique;
8 }
9
10 module.exports = { unsoldProducts };
```

np1p2p3...pnms1s2s3...sm

Output Format

Space-separated product IDs that were never sold

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq m \leq n$
- $0 \leq \text{productID} \leq 10^9$

Result

failed

Score: 0

Testcase

Result

Visible: 2

Hidden: 0

Submission Summary

All Tests Passed

Points: 50/50

Tests: 5/5

Accepted

Time: 0.17

Memory: 37800.00

Run

Submit

Department Salary Report

easy

50 Points

Problem

Given an array of employees, calculate the total salary for each department.

Return an object where:

key = department name

value = total salary

Input Format

N name1 department1 salary1 name2 department2 salary2 input: 4 A IT 1000 B IT 2000 C HR 3000 D HR 4000 ... nameN departmentN salaryN

Output Format

Print an object (dictionary/map) where: key = department value = total salary output: IT: 3000, HR: 7000

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq \text{salary} \leq 10^9$
- $1 \leq \text{length of name, department} \leq 50$

Result

failed

Score: 0

Code Editor

Language

javascript

Value: 354 chars

Reset Boilerplate

```
1 function departmentSalary(employees) {
2     // Write your code here
3     salary = {}
4     for (let obj of employees) {
5         if (salary[obj.department]) {
6             salary[obj.department] += obj.salary;
7         }
8         else {
9             salary[obj.department] = obj.salary;
10        }
11    }
12    return salary;
13 }
14
15 module.exports = { departmentSalary };
```

Testcase

Result

Visible: 3

Hidden: 0

Submission Summary

All Tests Passed

Points: 50/50

Tests: 5/5

Accepted

Time: 0.24

Memory: 37852.00

Run

Submit

Most Frequent Element

easy

50 Points

Array 3.0

Problem

Given an integer array nums, return the element that appears most frequently.

If multiple elements have the same frequency, return the smallest element among them.

Input Format

N num1 num2 num3 ... numN input: 5 1 2 2 3 3

Code Editor

Language

javascript

Value: 393 chars

Reset Boilerplate

```
1 function mostFrequentElement(nums) {
2     let maxCount = 0;
3     let result;
4     let frequency = {};
5
6
7     for (let num of nums) {
8         frequency[num] = (frequency[num] || 0) + 1;
9         if (frequency[num] > maxCount) {
10             maxCount = frequency[num];
11             result = num;
12         }
13     }
14 }
```

Output Format

Single integer → most frequent element Output: 2

Constraints

- $1 \leq N \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

Result

failed

Score: 0

```
13     }
14     return result;
15 }
16
17 module.exports = { mostFrequentElement };
```

Testcase

Result

Visible: 2

Hidden: 0

Submission Summary

All Tests Passed

Points: 50/50

Tests: 5/5

Accepted

Time: 0.22

Memory: 36044.00

Run

Submit

Close